

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
**«МИРЭА – Российский технологический университет»**

**РТУ МИРЭА**

---

**Институт информационных технологий (ИИТ)**
**Кафедра математического обеспечения и стандартизации информационных технологий(МОСИТ)**

# ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №1

по дисциплине «Технология разработки программных приложений»

Студент группы *ИНБО-12-23. Полиэктов М. А.* _____
(подпись)

Преподаватель *Петренко А.А.* _____
(подпись)

Москва 2025 г.

**Содержание**

Цель работы ..... 3
Часть 1. Основные команды Git ..... 4
Часть 2. Системы управления репозиториями ..... 12
Часть 3. Работа с ветвлением и оформление кода ..... 22
Ответы на вопросы ..... 26
Вывод ..... 28

## Цель работы

Получить навыки по работе с командной строкой и git’ом.

## Часть 1. Основные команды Git

Для начала работы с git’ом необходимо его установить и настроить. После установки настроим свое имя и email (рис. 1).

```
Polie@AUP3 MINGW64 ~
$ git config --list
diff.astextplain.textconv=astextplain
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.smudge=git-lfs smudge -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
http.sslbackend=schannel
core.autocrlf=true
core.fscache=true
core.symlinks=false
pull.rebase=false
credential.helper=manager
credential.https://dev.azure.com.usehttppath=true
init.defaultbranch=master
user.email=poliektovma@gmail.com
user.name=Maxim
core.autocrlf=true
core.safecrlf=warn
core.quotepath=off
```

Рисунок 1 – настройка git

Далее для начала работы создадим локальный репозиторий и добавим несколько файлов в него(рис. 2).

```
Polie@AUP3 MINGW64 ~
$ cd C:/Users/Polie/Mirea_source/Git_Project

Polie@AUP3 MINGW64 ~/Mirea_source/Git_Project ((b7360ad...))
$ touch proekt.html
```

Рисунок 2 – Создание репозитория и файла

Для изменения файла воспользуемся редактором nano. Вписываем свой текст в данный файл и нажимаем ctrl+x, для сохранения нажимаем Y, а далее Enter. (рис. 3).




Рисунок 3 – Редактор nano

Теперь добавим файл в репозиторий и проверим итоговый статус.

```
Polie@AUP3 MINGW64 ~/Mirea_source/Git_Project ((b7360ad...))
$ git init
Reinitialized existing Git repository in C:/Users/Polie/Mirea_source/Git_Project
/.git/

Polie@AUP3 MINGW64 ~/Mirea_source/Git_Project ((b7360ad...))
$ git add proekt.html
warning: in the working copy of 'proekt.html', LF will be replaced by CRLF the n
ext time Git touches it

Polie@AUP3 MINGW64 ~/Mirea_source/Git_Project ((b7360ad...))
$ git commit -m "Добавление первого файла"
[detached HEAD a8bfb7a] Добавление первого файла
 1 file changed, 1 insertion(+)
 create mode 100644 proekt.html

Polie@AUP3 MINGW64 ~/Mirea_source/Git_Project ((a8bfb7a...))
$ git status
HEAD detached from baf2ab4
nothing to commit, working tree clean
```

Рисунок 4 – Инициализация репозитория и добавление файла

Проиндексируем изменения и проверим состояние (рис. 5).

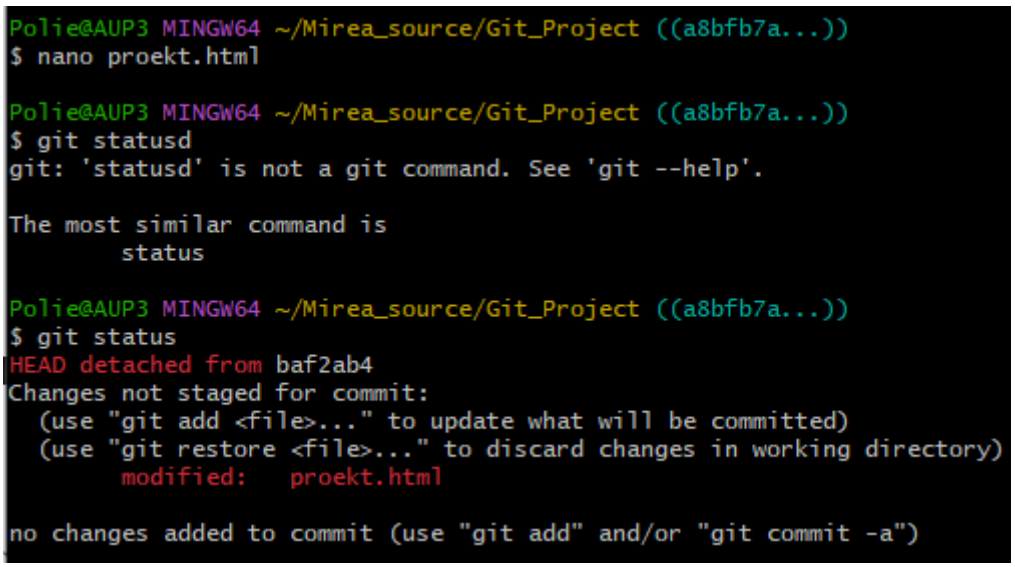



Рисунок 5 – Индексация изменений

Зафиксируем изменения (рис. 6).

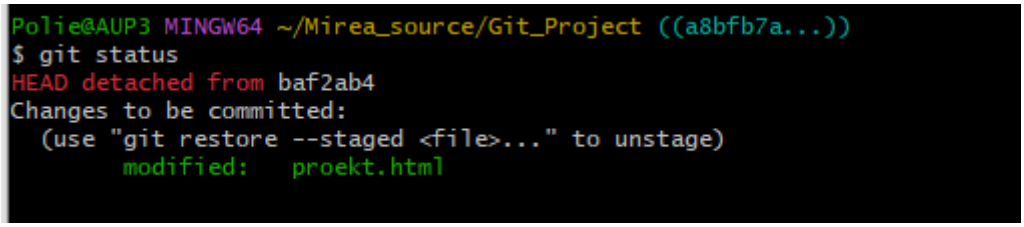



Рисунок 6 – фиксирование изменений

Теперь попытаемся последовательно изменить один файл 2 раза (рис. 7). Из рисунка видно, что первое изменение можно закоммитить, а второе – нет.

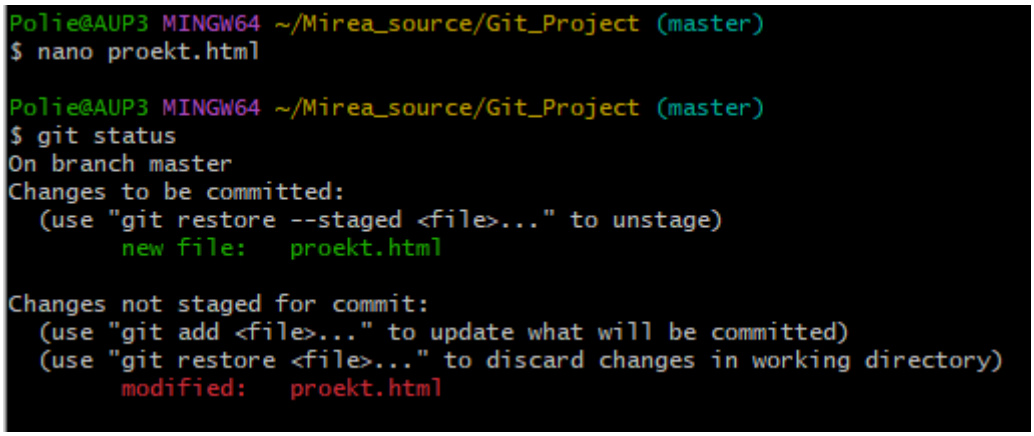



Рисунок 7 – Изменение одного файла несколько раз

Добавим коммит(рис. 8).

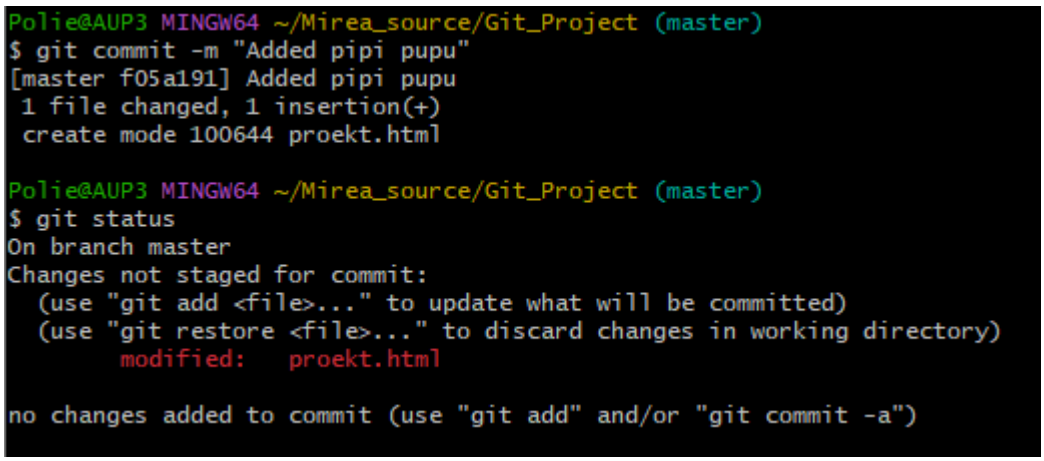



Рисунок 8 – Статус

Один из файлов не был закоммичен, исправим это следующими командами:

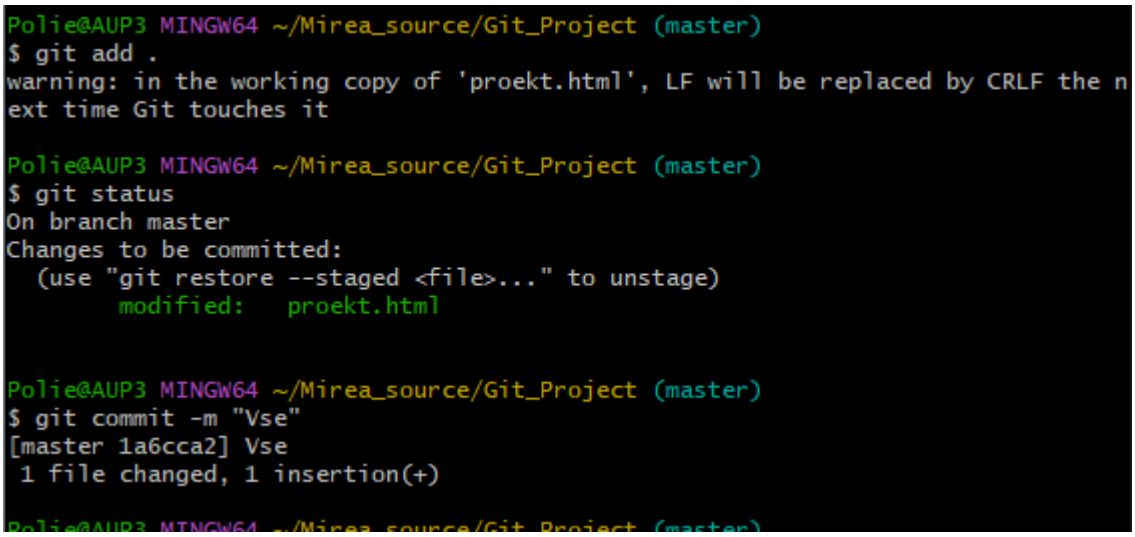



Рисунок 9 – Коммит файла

Просмотрим историю коммитов.

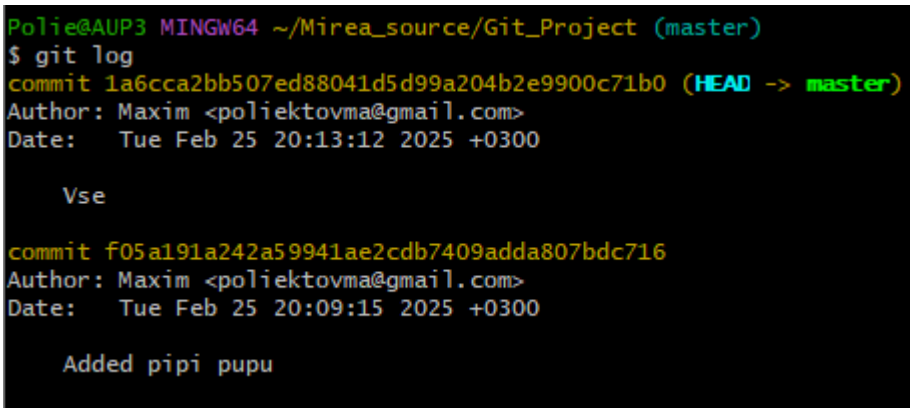



Рисунок 10 – История коммитов

Теперь вернемся на старую версию. Для этого нам нужно знать хэши(рис. 11).

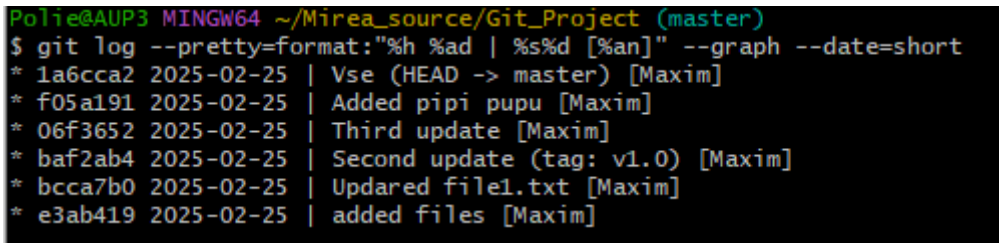



Рисунок 11 – История коммитов

Перейдем на предпоследнюю версию с хэш-кодом f05a191.

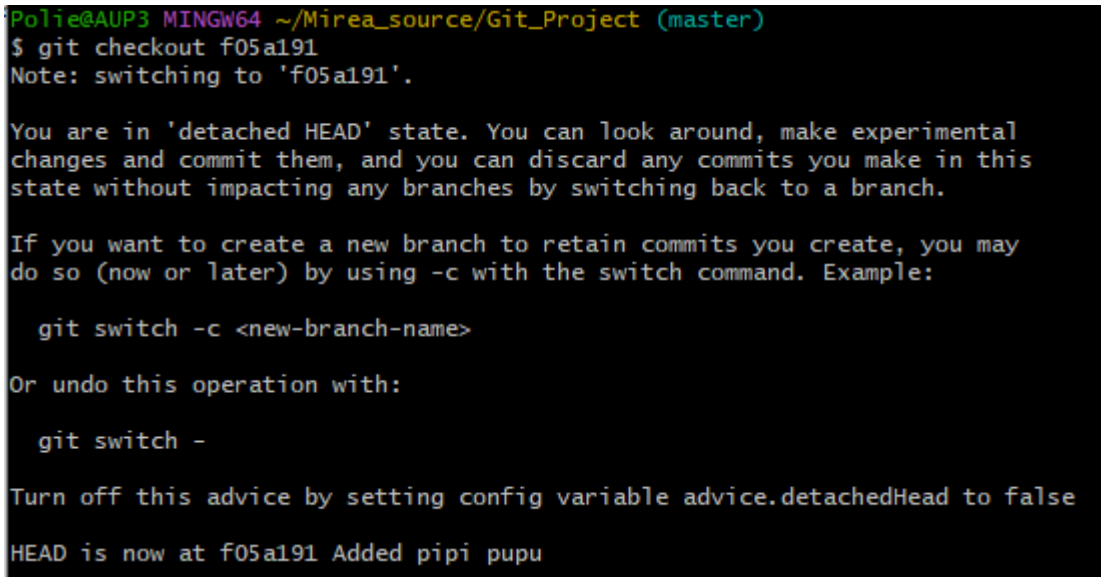



Рисунок 12 – Перемещение на другую версию

Посмотри, что содержится в файле в текущей версии. Перейдем на новейшую версию и посмотрим содержимое файла.

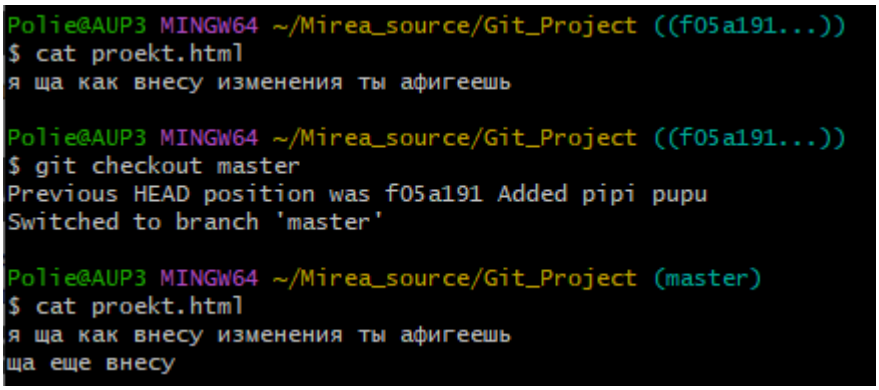



Рисунок 13 – Различие в версиях

Теперь отменим изменения в файле до  индексации.

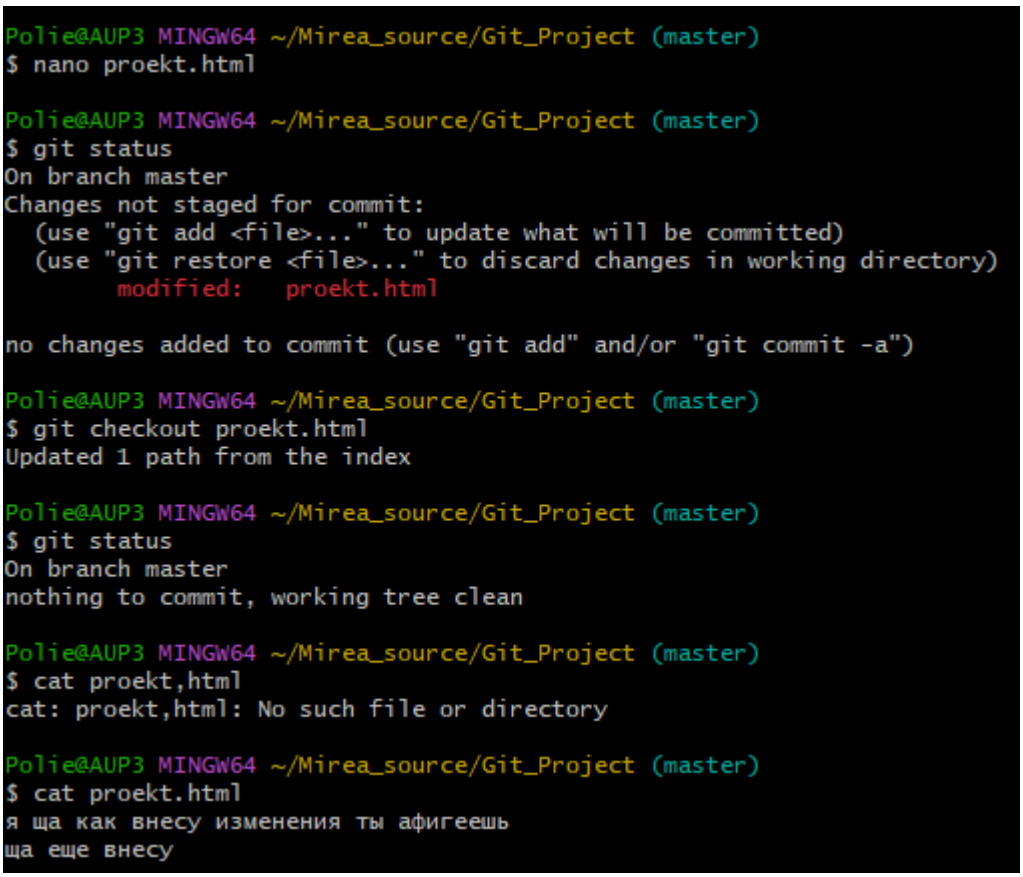



Рисунок 14 – Отмена последних изменений до индексации

Теперь отменим изменения в файле после индексации и  до коммита.

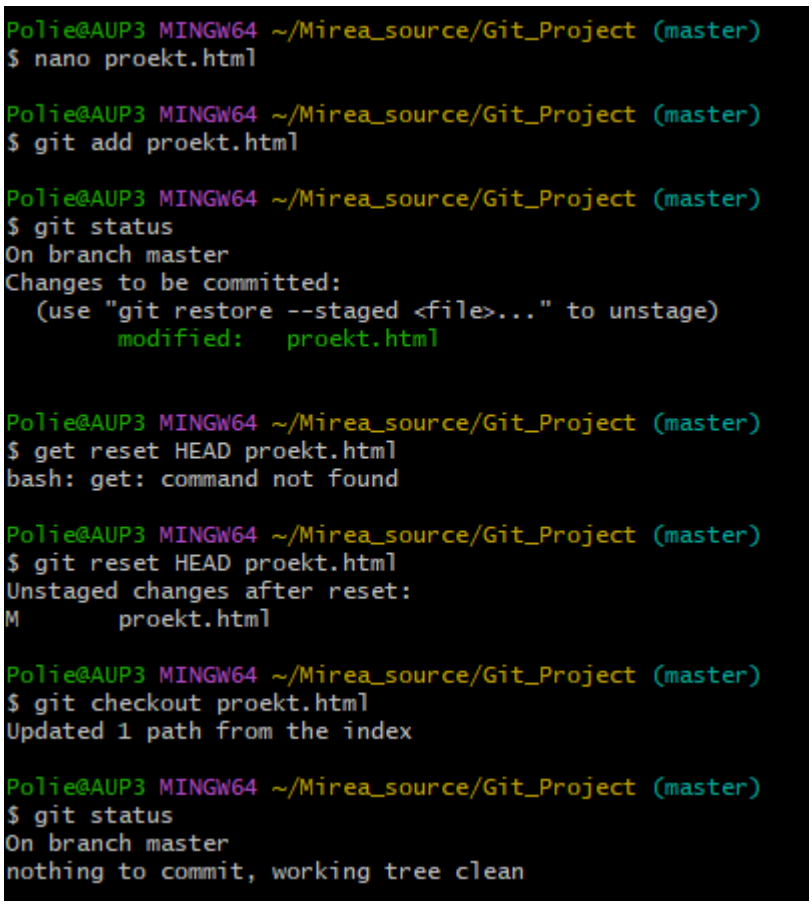



Рисунок 15 – Отмена изменений до после индексации и до коммита

Теперь отменим изменения после коммита.

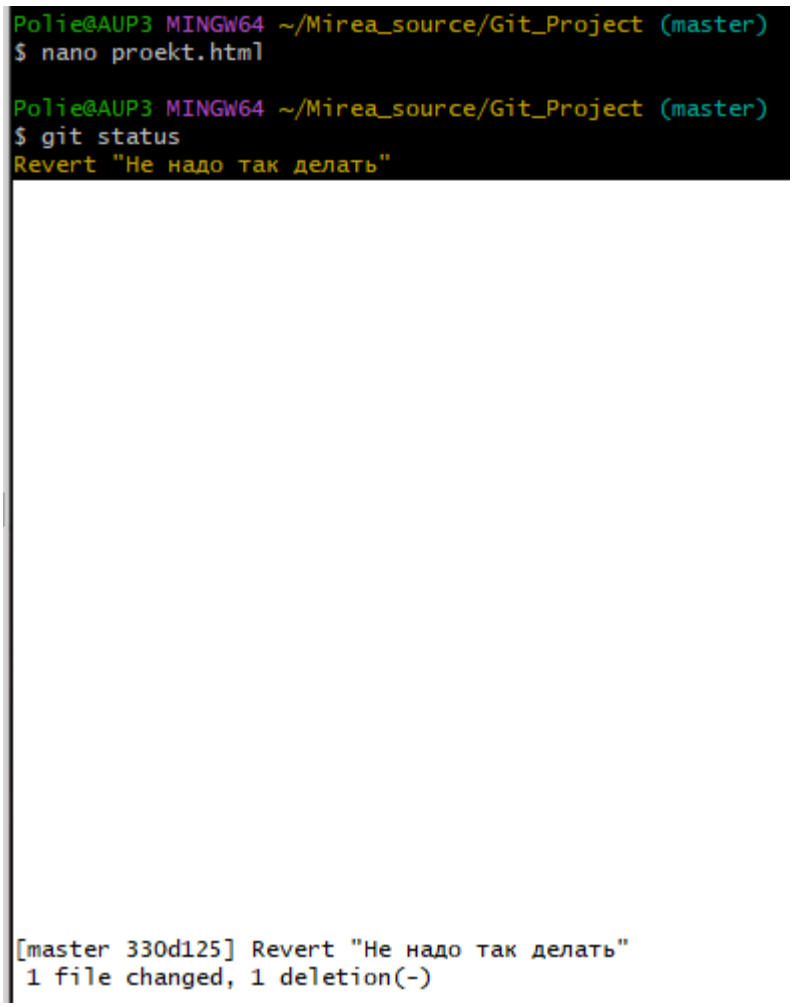



Рисунок 16 – Отмена коммита

## Часть 2. Системы управления репозиториями

GitHub — веб-сервис, который основан на системе Git. Это такая социальная сеть для разработчиков, которая помогает удобно вести коллективную разработку IT-проектов. Здесь можно публиковать и редактировать свой код, комментировать чужие наработки, следить за новостями других пользователей. Именно в GitHub работаем мы, команда Академии, и студенты интенсивов.

Чтобы работать со своего компьютера с GitHub, иметь доступ к проектам, хранящимся на сервисе, выполнять команды в консоли без постоянного подтверждения пароля, нужно пройти авторизацию у сервера. В этом помогают SSH-ключи.

Каждый SSH-ключ содержит пару: открытый (публичный) и закрытый (приватный) ключ. Открытый ключ отправляется на сервер, его можно не прятать от всех и не переживать, что кто-то его увидит и украдёт. Он бесполезен без своей пары — закрытого ключа. А вот закрытый ключ — секретная часть. Доступ к нему должен быть только у вас.

Вы отправляете какую-то информацию на сервер, где хранится ваш публичный ключ, сервер понимает, что вы это вы, то есть идентифицирует именно вас, и даёт вам какой-то ответ. И только вы можете расшифровать этот ответ, потому что только у вас есть подходящий закрытый ключ. Получается что-то вроде связки логин-пароль только намного безопасней. Ваш пароль кто-то может узнать или подобрать, а чтобы получить ваш приватный SSH-ключ, злоумышленнику придётся взломать ваш компьютер.

Чтобы пройти авторизацию по SSH-ключу, его надо сгенерировать или найти уже ранее созданный ключ на своём компьютере.

Сначала проверим, есть ли уже на компьютере ключ. По умолчанию SSH-ключи хранятся в каталоге ~/.ssh, поэтому нужно проверить содержимое этого каталога.

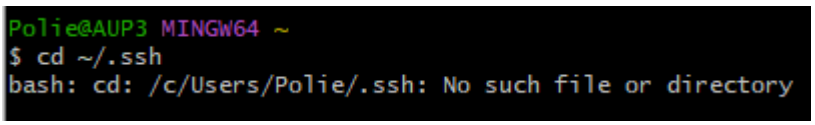



Рисунок 17 – Проверка наличия ssh-ключа

Создадим новый ключ.

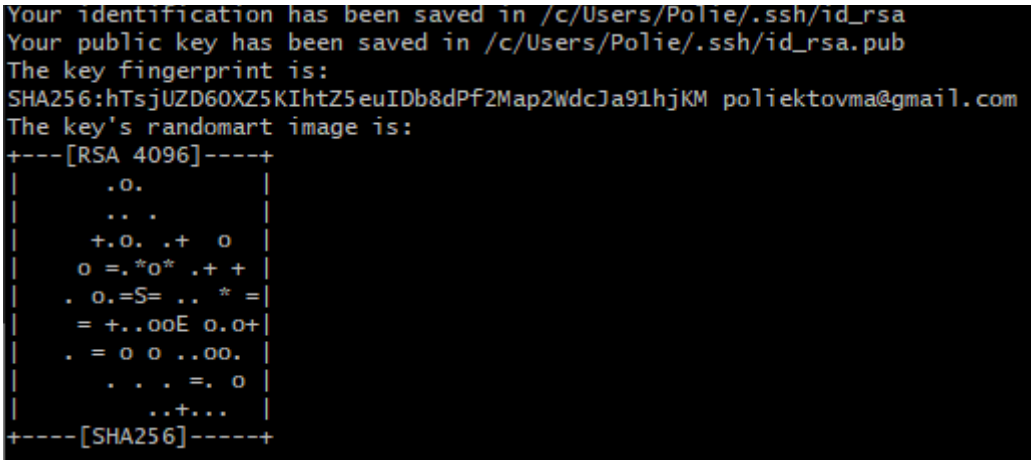



Рисунок 18 – Новый ssh-ключ

Добавим ключ в ssh-agent.

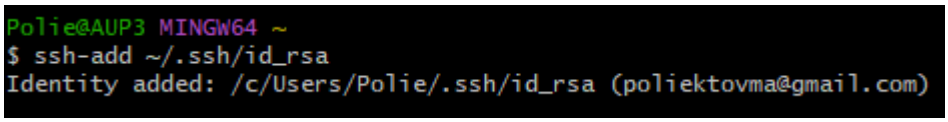



Рисунок 19 – Добавление ключа в ssh-agent

Теперь свяжем локальный репозиторий и удаленный на сайте github.

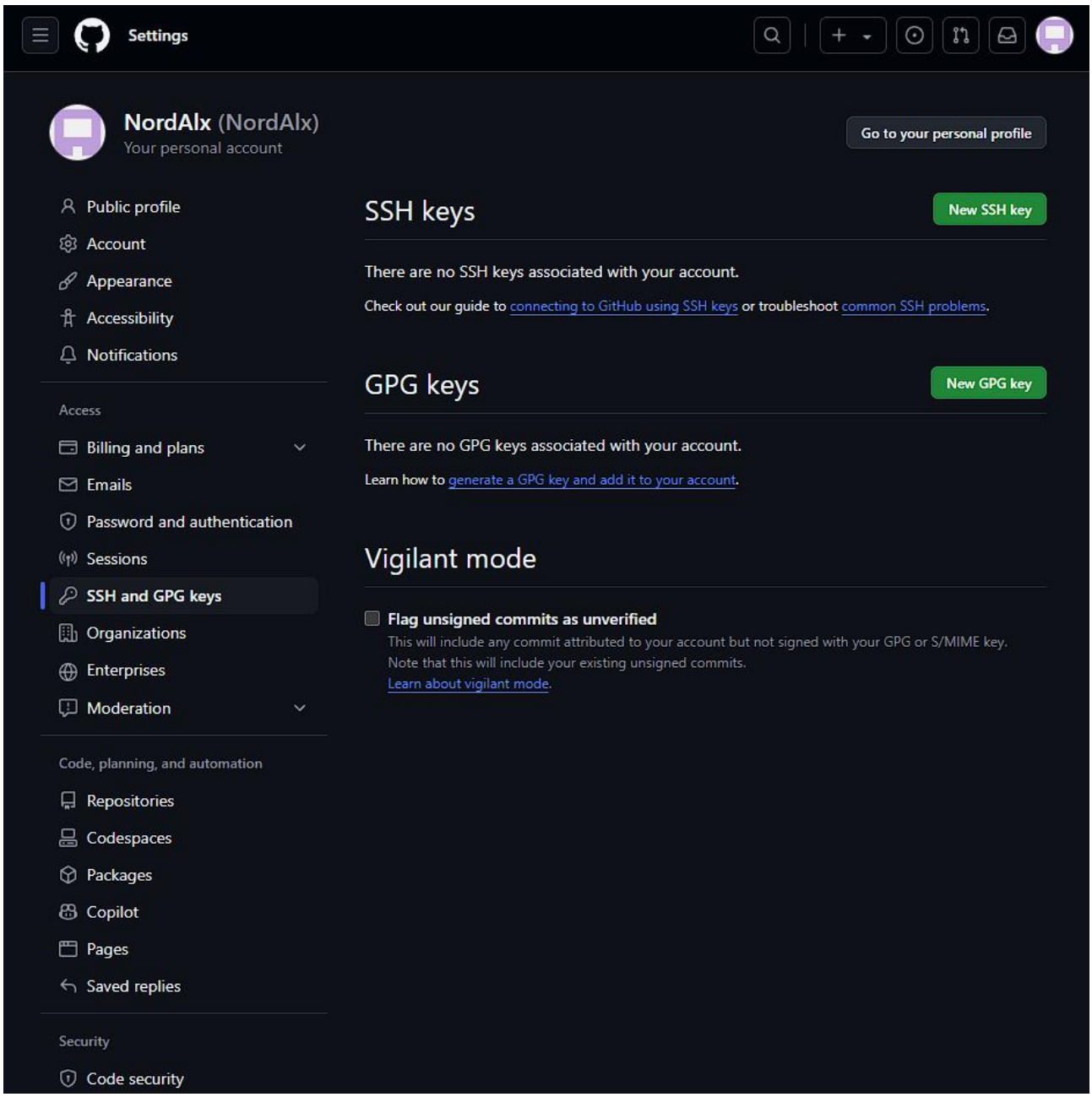



Рисунок 20 – Окно работы с ключами

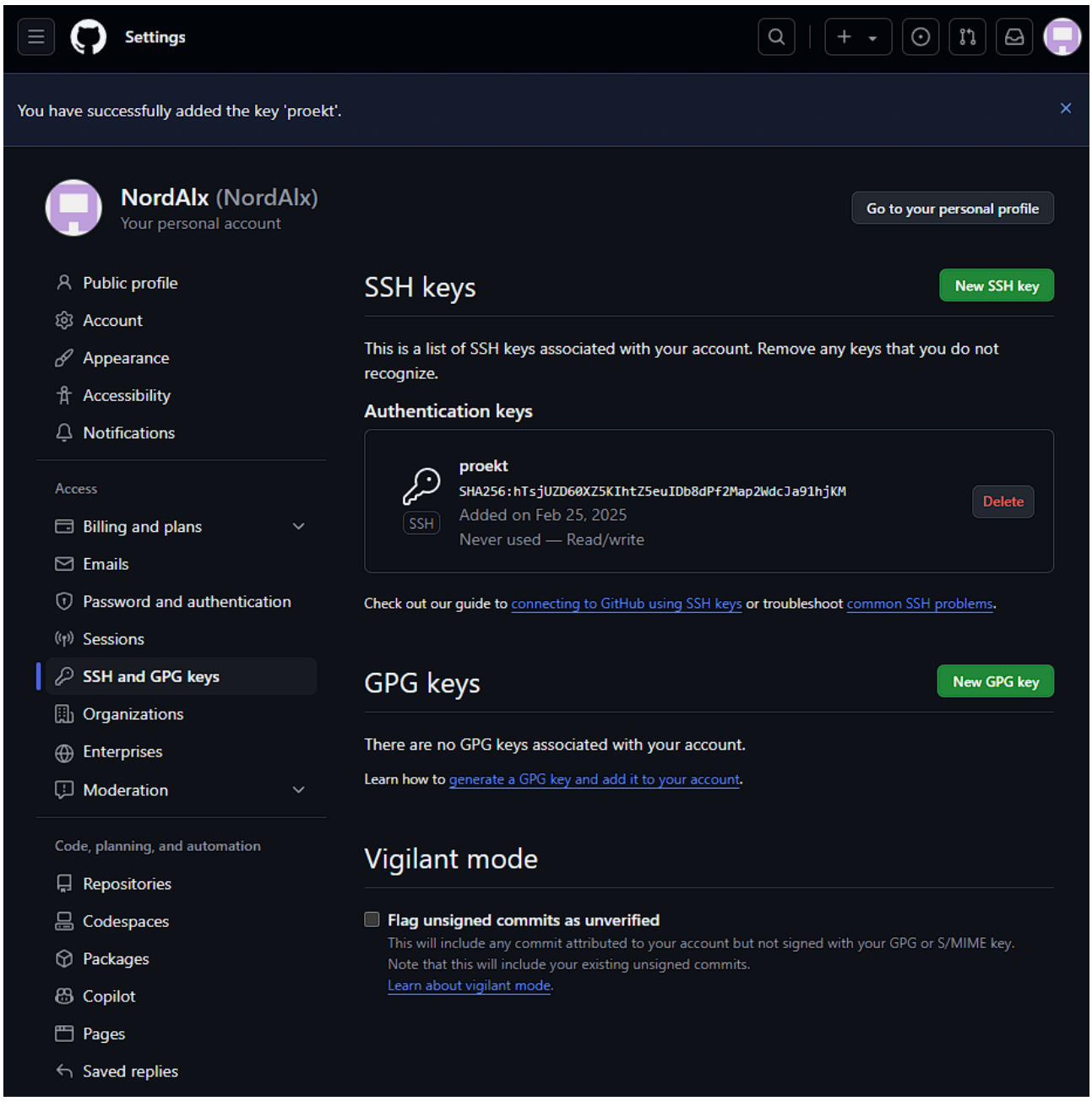



Рисунок 21 – Новый ключ

Теперь создадим новый репозиторий и свяжем его с репозиторией на github.

```
Polie@AUP3 MINGW64 ~
$ cd C:/Users/Polie/Mirea_source/Local_rep

Polie@AUP3 MINGW64 ~/Mirea_source/Local_rep
$ git init
Initialized empty Git repository in C:/Users/Polie/Mirea_source/Local_rep/.git/

Polie@AUP3 MINGW64 ~/Mirea_source/Local_rep (master)
$ touch readme.txt

Polie@AUP3 MINGW64 ~/Mirea_source/Local_rep (master)
$ git remote add project git@github.com:NordAlx/proekt.git
```

Рисунок 22 – Создание и привязка репозитория

Создадим новую ветку и назовем ее main.

```
Polie@AUP3 MINGW64 ~/Mirea_source/Local_rep (master)
$ git checkout -b main
Switched to a new branch 'main'

Polie@AUP3 MINGW64 ~/Mirea_source/Local_rep (main)
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        readme.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Рисунок 23 – Создание и переход на новую ветку

Теперь сольем вместе обе ветки: not_main и main.

```
Polie@AUP3 MINGW64 ~
$ cd C:/Users/Polie/Mirea_source/Local_rep

Polie@AUP3 MINGW64 ~/Mirea_source/Local_rep (main|MERGING)
$ git merge not_main
fatal: You have not concluded your merge (MERGE_HEAD exists).
Please, commit your changes before you merge.

Polie@AUP3 MINGW64 ~/Mirea_source/Local_rep (main|MERGING)
$ git commit -m "Слияние"
[main 9ee83c2] Слияние
```

Рисунок 24 – Слияние веток

Выполним задание индивидуального варианта:

1) Клонируйте непустой удаленный репозиторий на локальную машину

2) Создайте новую ветку и выведите список всех веток

3) Произведите 3 коммита в новой ветке в разные файлы

4) Выгрузите изменения в удаленный репозиторий

5) Добейтесь того, чтобы эти три изменения были представлены одним коммитом

6) Выведите в консоли различия между веткой master и новой веткой

7) Слейте новую ветку с master при помощи merge

Клонируем репозиторий с github в отдельную папку.

```
Polie@AUP3 MINGW64 ~/Mirea_source/Local_rep (main)
$ cd C:/Users/Polie/Mirea_source

Polie@AUP3 MINGW64 ~/Mirea_source
$ git clone https://github.com/NordAlx/Rep_to_clone
Cloning into 'Rep_to_clone'...
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 6 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (6/6), done.
```

Рисунок 25 – Клонирование удаленного репозитория

Перейдем в репозиторий и создадим новую ветку.

```
Polie@AUP3 MINGW64 ~/Mirea_source
$ cd C:/Users/Polie/Mirea_source/Rep_to_clone

Polie@AUP3 MINGW64 ~/Mirea_source/Rep_to_clone (main)
$ git checkout -b new_branch
Switched to a new branch 'new_branch'

Polie@AUP3 MINGW64 ~/Mirea_source/Rep_to_clone (new_branch)
$ git branch
  main
* new_branch
```

Рисунок 26 – Создание новой ветки

Произведем коммиты  в новой ветке в файлах.

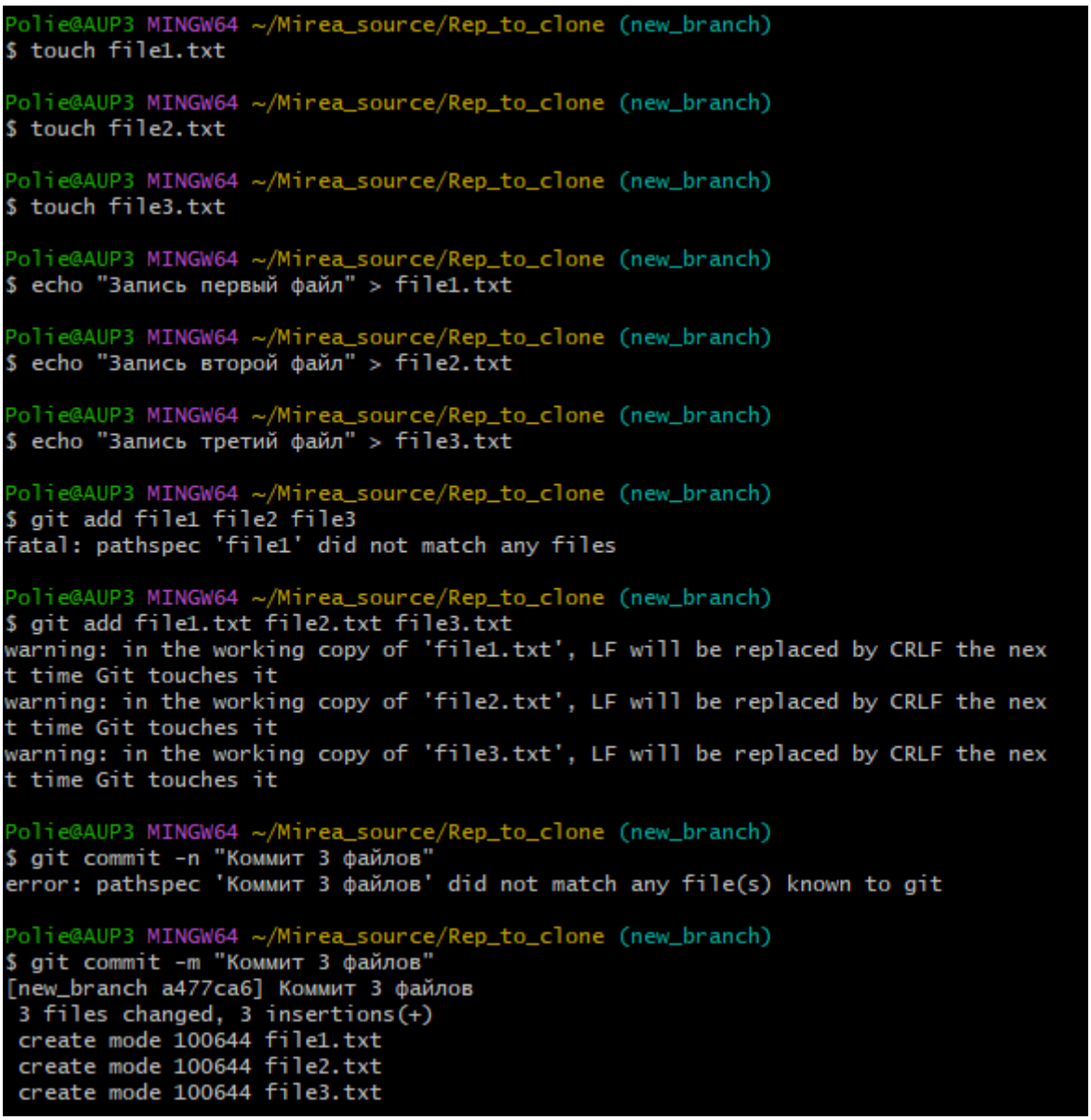



Рисунок 27 – Коммиты в новой ветке в новых файлах

Теперь подключимся к удаленному репозиторию и выгрузим новую ветку

```
Polie@AUP3 MINGW64 ~/Mirea_source/Rep_to_clone (new_branch)
$ git remote
origin

Polie@AUP3 MINGW64 ~/Mirea_source/Rep_to_clone (new_branch)
$ git remote set-url origin git@github.com:NordAlx/Rep_to_clone.git

Polie@AUP3 MINGW64 ~/Mirea_source/Rep_to_clone (new_branch)
$ ssh -T git@github.com
The authenticity of host 'github.com (140.82.121.3)' can't be established.
ED25519 key fingerprint is SHA256:+DiY3wvvV6TuJJhbpZisF/zLDAOzPMSvHdkr4UvCOqU.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'github.com' (ED25519) to the list of known hosts.
Hi NordAlx! You've successfully authenticated, but GitHub does not provide shell
 access.

Polie@AUP3 MINGW64 ~/Mirea_source/Rep_to_clone (new_branch)
$ git push origin new_branch
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (5/5), 482 bytes | 482.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'new_branch' on GitHub by visiting:
remote:      https://github.com/NordAlx/Rep_to_clone/pull/new/new_branch
remote:
To github.com:NordAlx/Rep_to_clone.git
 * [new branch]      new_branch -> new_branch
```

Рисунок 28 – Выгрузка новой ветки

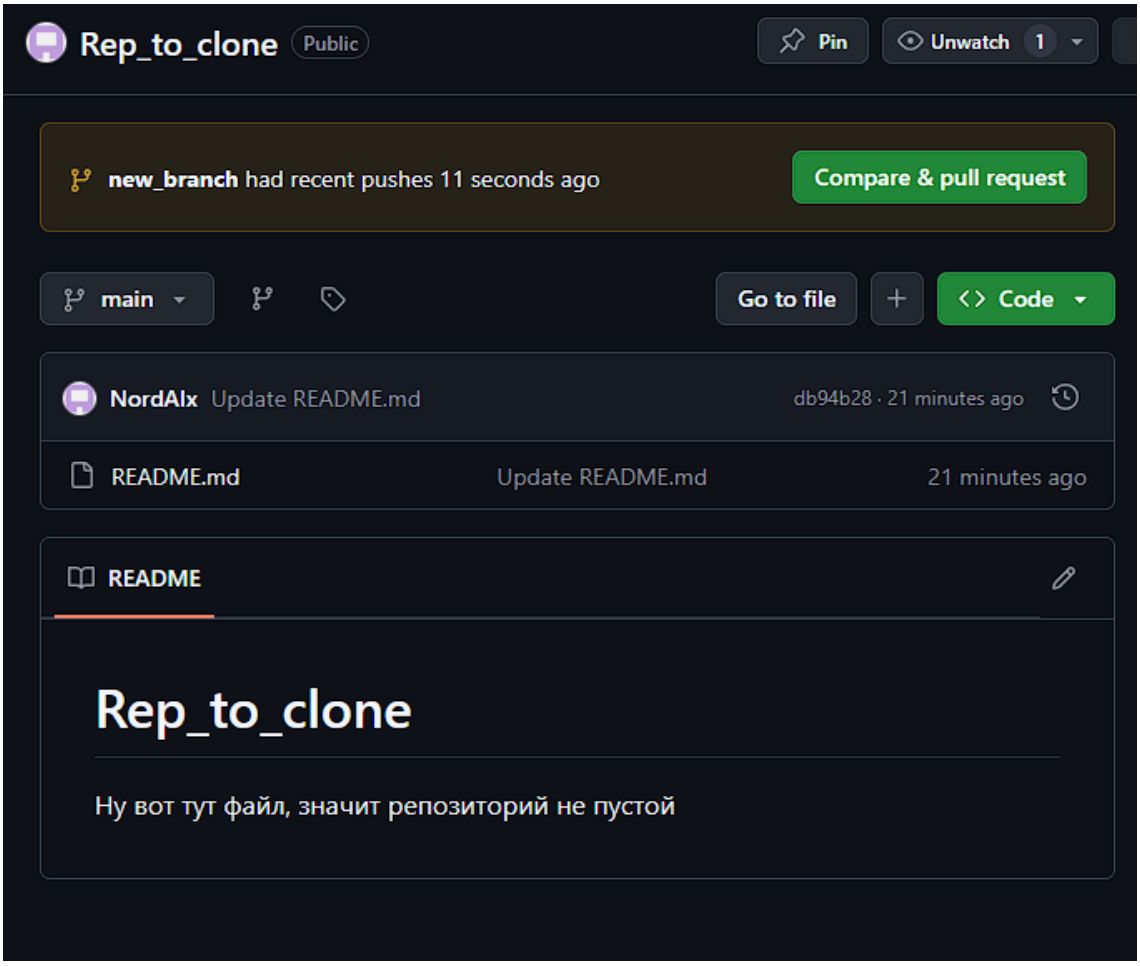



Рисунок 29 – Результат

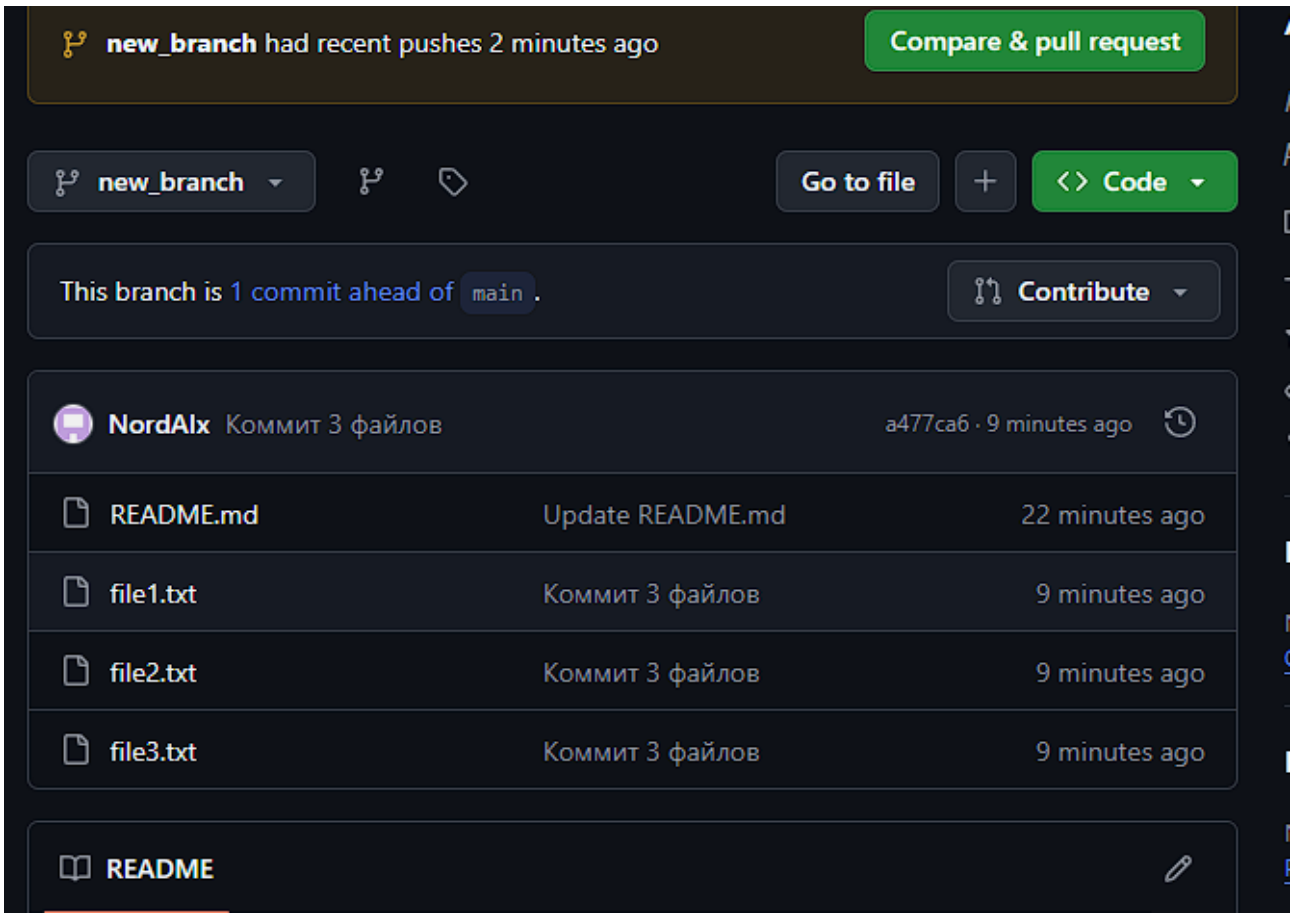



Рисунок 30 – Выгрузка одним коммитом

Выведем разницу между ветками.

```
Polie@AUP3 MINGW64 ~/Mirea_source/Rep_to_clone (new_branch)
$ git diff main new_branch
diff --git a/file1.txt b/file1.txt
new file mode 100644
index 0000000..af60205
--- /dev/null
+++ b/file1.txt
@@ -0,0 +1 @@
+Запись первый файл
diff --git a/file2.txt b/file2.txt
new file mode 100644
index 0000000..36aefff
--- /dev/null
+++ b/file2.txt
@@ -0,0 +1 @@
+Запись второй файл
diff --git a/file3.txt b/file3.txt
new file mode 100644
index 0000000..92ca85e
--- /dev/null
+++ b/file3.txt
@@ -0,0 +1 @@
+Запись третий файл
```

Рисунок 31 – Вывод различий в ветках

И наконец сольем две ветки вместе.

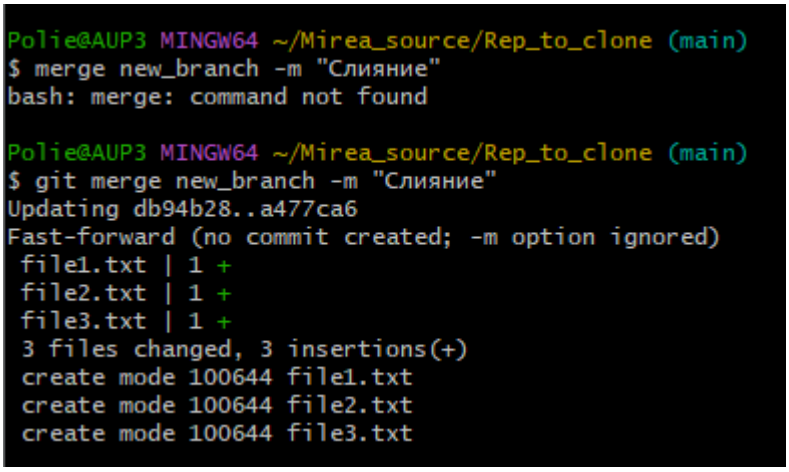



Рисунок 32 – Слияние веток

## Часть 3. Работа с ветвлением и оформление кода

Сделаем форк репозитория и клонируем его.

```
Polie@AUP3 MINGW64 ~/Mirea_source/Rep_to_clone (main)
$ cd C:/Users/Polie/Mirea_source

Polie@AUP3 MINGW64 ~/Mirea_source
$ git clone git@github.com:NordAlx/model_search.git
Cloning into 'model_search'...
remote: Enumerating objects: 274, done.
remote: Counting objects: 100% (86/86), done.
remote: Compressing objects: 100% (35/35), done.
remote: Total 274 (delta 59), reused 51 (delta 51), pack-reused 188 (from 1)
Receiving objects: 100% (274/274), 272.43 KiB | 1.04 MiB/s, done.
Resolving deltas: 100% (134/134), done.

Polie@AUP3 MINGW64 ~/Mirea_source
$ cd ~model_search
bash: cd: ~model_search: No such file or directory

Polie@AUP3 MINGW64 ~/Mirea_source
$ cd C:/Users/Polie/Mirea_source/model_search
```

Рисунок 33 – Клонирование форки

Создадим 2 ветки.

```
Polie@AUP3 MINGW64 ~/Mirea_source/model_search (master)
$ git checkout -b branch1
Switched to a new branch 'branch1'

Polie@AUP3 MINGW64 ~/Mirea_source/model_search (branch1)
$ git checkout -b branch2
Switched to a new branch 'branch2'

Polie@AUP3 MINGW64 ~/Mirea_source/model_search (branch2)
$ git branch
  branch1
* branch2
  master
```

Рисунок 34 – Создание новых веток

Произведем 3 коммита в этихих ветках.

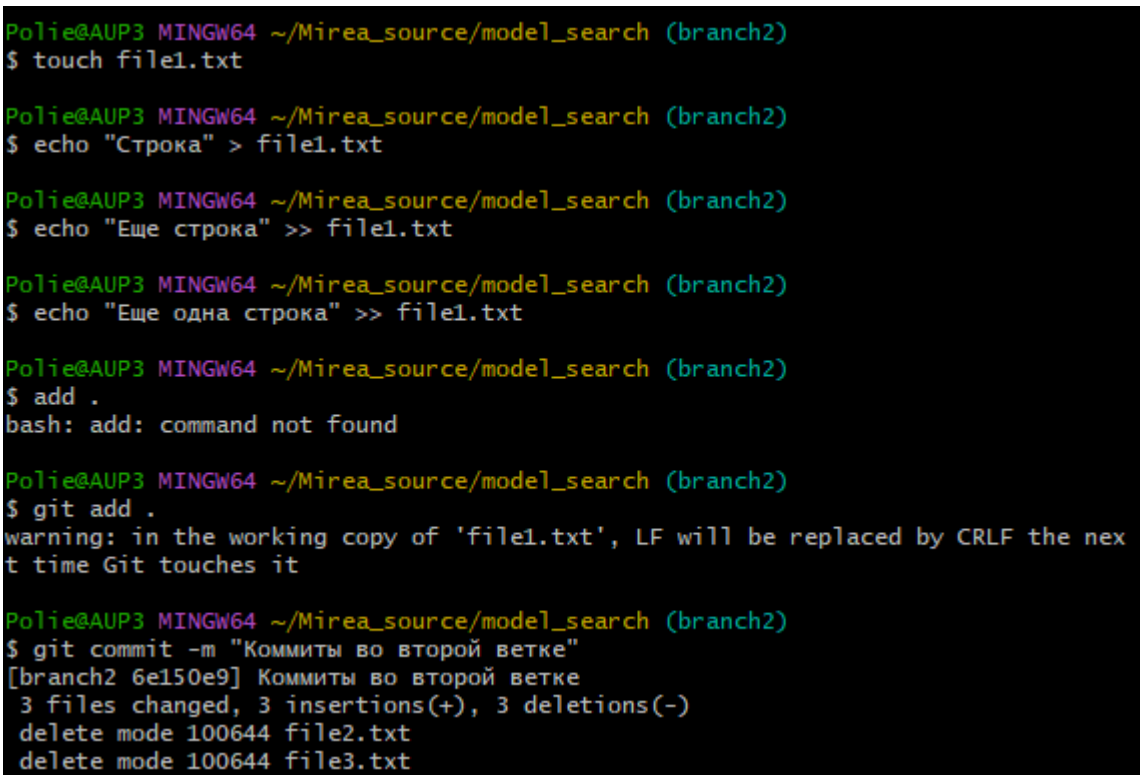



Рисунок 35 – Коммиты в одной из веток

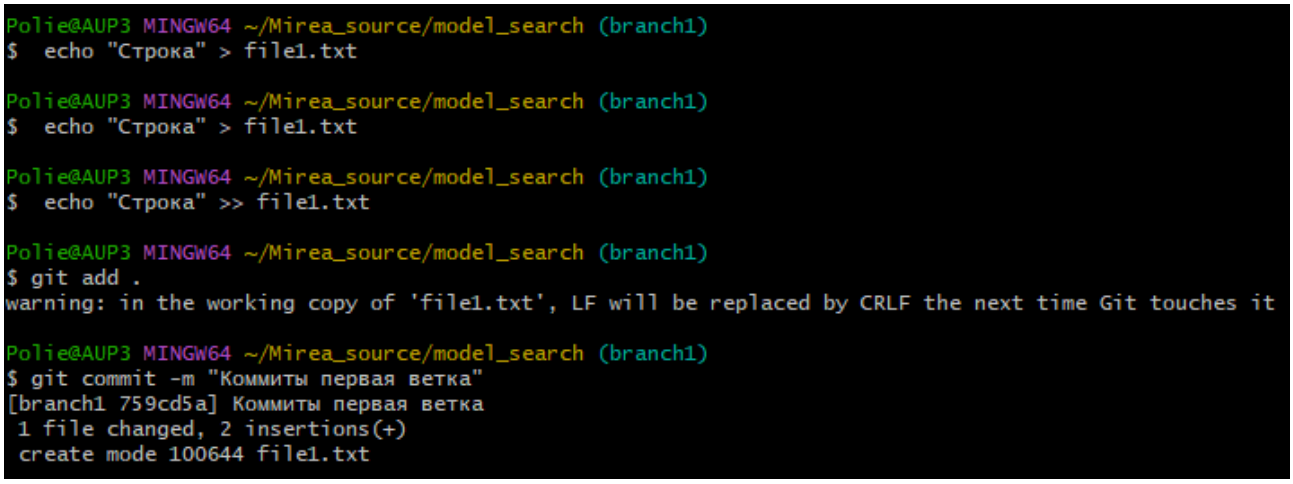



Рисунок 36 – Коммиты в другой ветке

Теперь сольем вместе обе ветки, разрешив конфликт врчуную.

```
Polie@AUP3 MINGW64 ~/Mirea_source/model_search (branch1)
$ git checkout branch2
Switched to branch 'branch2'

Polie@AUP3 MINGW64 ~/Mirea_source/model_search (branch2)
$ git merge branch1
Auto-merging file1.txt
CONFLICT (add/add): Merge conflict in file1.txt
Automatic merge failed; fix conflicts and then commit the result.

Polie@AUP3 MINGW64 ~/Mirea_source/model_search (branch2|MERGING)
$ git add conflicting-file1.txt
fatal: pathspec 'conflicting-file1.txt' did not match any files

Polie@AUP3 MINGW64 ~/Mirea_source/model_search (branch2|MERGING)
$ git add file1.txt

Polie@AUP3 MINGW64 ~/Mirea_source/model_search (branch2|MERGING)
$ git commit -m "merged A and B"
[branch2 74b12c3] merged A and B
```

Рисунок 37 – Слияние 2 веток

Выгрузим все изменения в репозиторий.

```
Polie@AUP3 MINGW64 ~/Mirea_source/model_search (branch2)
$ git push origin branch1
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 333 bytes | 333.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'branch1' on GitHub by visiting:
remote:      https://github.com/NordAlx/model_search/pull/new/branch1
remote:
To github.com:NordAlx/model_search.git
 * [new branch]      branch1 -> branch1

Polie@AUP3 MINGW64 ~/Mirea_source/model_search (branch2)
$ git push origin branch2
Enumerating objects: 14, done.
Counting objects: 100% (14/14), done.
Delta compression using up to 4 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (11/11), 1.02 KiB | 348.00 KiB/s, done.
Total 11 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (4/4), completed with 1 local object.
remote:
remote: Create a pull request for 'branch2' on GitHub by visiting:
remote:      https://github.com/NordAlx/model_search/pull/new/branch2
remote:
To github.com:NordAlx/model_search.git
 * [new branch]      branch2 -> branch2
```

Рисунок 38 – Выгрузка изменений

Скопируем репозиторий еще раз и сделаем 3 коммита в branch1

```
Polie@AUP3 MINGW64 ~/Mirea_source/OneMore/model_search (branch1)
$ echo "Ну скока можно" > file3

Polie@AUP3 MINGW64 ~/Mirea_source/OneMore/model_search (branch1)
$ echo "Ну скока можно" >> file3

Polie@AUP3 MINGW64 ~/Mirea_source/OneMore/model_search (branch1)
$ echo "Ну скока можно" >> file3

Polie@AUP3 MINGW64 ~/Mirea_source/OneMore/model_search (branch1)
$ git add .
warning: in the working copy of 'file3', LF will be replaced by CRLF the ne
me Git touches it

Polie@AUP3 MINGW64 ~/Mirea_source/OneMore/model_search (branch1)
$ git commit -m "Да все"
[branch1 2c46aca] Да все
 1 file changed, 3 insertions(+)
 create mode 100644 file3
```

Рисунок 39 – Еще 3 коммита

Выгрузим все изменения.

```
Polie@AUP3 MINGW64 ~/Mirea_source/OneMore/model_search (branch1)
$ git push origin branch1
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 309 bytes | 309.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:NordAlx/model_search.git
   759cd5a..2c46aca  branch1 -> branch1
```

Рисунок 40 – Выгрузка в удаленный репозиторий

Теперь выгрузим все изменения из старого клона с параметром --force.

```
Polie@AUP3 MINGW64 ~/Mirea_source/model_search (branch1)
$ git push origin branch1 --force
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 331 bytes | 331.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:NordAlx/model_search.git
 + 2c46aca...9f9d4b6 branch1 -> branch1 (forced update)
```

Рисунок 41 – Принудительная выгрузка

Теперь получим все изменения.

```
Polie@AUP3 MINGW64 ~/Mirea_source/model_search (branch1)
$ git pull origin branch1
From github.com:NordAlx/model_search
 * branch            branch1    -> FETCH_HEAD
Already up to date.
```

Рисунок 42 – Все изменения

## Ответы на вопросы

### 1. Какие существуют типы систем контроля версий?

Системы контроля версий (VCS) бывают трех типов:

1. Локальные системы контроля версий (Local VCS)

Хранят историю изменений файлов на одном компьютере. Пример: RCS (Revision Control System).

2. Централизованные системы контроля версий (Centralized VCS, CVCS)

Все версии файлов хранятся на одном удалённом сервере, и каждый пользователь получает доступ к копии. Примеры: SVN (Subversion), Perforce, CVS, TFS (Team Foundation Server).

3. Распределённые системы контроля версий (Distributed VCS, DVCS)

Каждый разработчик имеет полную копию репозитория с историей изменений, а не только последнюю версию. Примеры: Git, Mercurial, Fossil.

### 2. К какому типу систем контроля версий относится Git?

Git — это распределённая система контроля версий (DVCS)**.** Каждый разработчик имеет полную копию репозитория и может работать без подключения к серверу.

### 3. Что такое репозиторий Git?

Репозиторий — это хранилище проекта в Git, которое включает файлы, папки и историю изменений.

### 4. Что такое коммит?

Коммит (commit) — это сохранённое состояние проекта в Git с комментарием.

### 5. Что такое ветка в репозитории Git?

Ветка (branch) — это отдельная линия разработки, которая позволяет работать над новой функциональностью, не затрагивая основную.

**6. Что такое тег в репозитории Git?**

Тег (tag) — это фиксированная метка на конкретном коммите (обычно для версии проекта).

**7. Что такое слияние двух веток?**

Слияние (merge) — это объединение изменений из одной ветки в другую.

**8. Что делает команда git add?**

Команда git add добавляет изменения в индекс (staging area)**,** чтобы они могли быть зафиксированы (commit) в следующем коммите.

## Вывод

Я получил навыки работы с командной строкой и git’ом.